Post-Mortem Report

Enterprise DevSecOps Security Lab - Lessons Learned, Residual Risk, and Improvement Roadmap

Document	Post-Mortem Document
Project	Enterprise DevSecOps Security Lab
Author	Jagriti Banerjee
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, ArgoCD, Falco, Prometheus, Grafana
Status	Portfolio-ready documentation

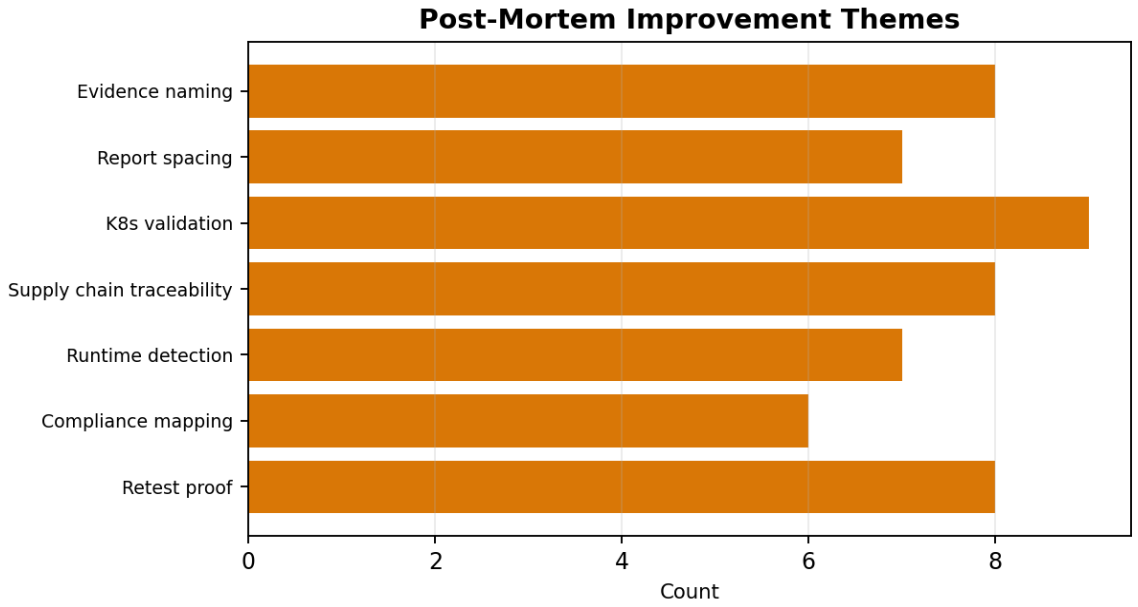
This document is written as professional portfolio evidence for a private enterprise-style DevSecOps lab. It is not a claim of formal certification, compliance attestation, or third-party audit approval.

Document quality controls: structured scope, evidence traceability, control mapping, validation criteria, residual risk notes, and executive-ready summary sections.

1. Executive Summary

This post-mortem reviews the Enterprise DevSecOps Security Lab as a completed portfolio project. The goal is to capture what was built, what worked, what required iteration, what risks remain, and how the project can be improved for future security engineering, DevSecOps, Kubernetes security, and cloud-native security interviews.

The lab successfully demonstrated vulnerable application testing, CI/CD security scanning, Docker and Kubernetes hardening, GitOps, supply-chain security, runtime detection, observability, compliance mapping, and professional reporting. The largest improvement area was document layout consistency and evidence normalization, which were addressed through iterative PDF rebuilds and clearer artifact indexing.



2. Project Objectives

- Build an end-to-end private DevSecOps lab without targeting external systems.
- Create a deliberately vulnerable Flask application and document real AppSec findings.
- Integrate security gates into CI/CD using SAST, SCA, Trivy, and Scorecard-style posture checks.
- Harden a Docker image and Kubernetes deployment using practical defense-in-depth controls.
- Use GitOps to enforce desired state and demonstrate drift remediation.
- Add supply-chain controls using Cosign, Syft, SBOMs, and provenance attestations.
- Simulate runtime and RBAC attacks, detect them with Falco, and validate remediation.
- Produce professional documentation suitable for recruiter and technical reviewer inspection.

3. What Went Well

Area	Positive Outcome	Why It Matters
Hands-on implementation	The project moved beyond theory into commands, YAML, scans, and evidence.	Demonstrates practical engineering ability.
Security breadth	The lab covered AppSec, CI/CD, containers, Kubernetes, GitOps, supply chain, runtime detection, and compliance.	Shows end-to-end DevSecOps thinking.

Area	Positive Outcome	Why It Matters
Attack plus remediation	The project included both exploit/detection proof and remediation validation.	Mimics real security assessment workflow.
Evidence-driven reporting	Screenshots and reports were tied to controls and findings.	Makes the project reviewable and credible.
Supply-chain maturity	Cosign, Syft, SBOM, provenance, Gatekeeper, and Scorecard concepts were added.	Aligns with modern software supply-chain priorities.
Runtime observability	Falco, Falcosidekick, Prometheus, and Grafana converted detections into dashboard and alerting evidence.	Shows detection engineering capability.

4. Challenges and Root Causes

Challenge	Root Cause	Correction / Lesson
Limited VM resources	Ubuntu VM had constrained RAM and disk for Kubernetes, ArgoCD, Falco, Prometheus, and Grafana.	Used one-node Kind cluster, swap, and stopped heavy components when not needed.
NetworkPolicy enforcement confusion	Kind default CNI may not enforce NetworkPolicy.	Recreated cluster with Calico for real allow/deny proof.
Report spacing issues	Text-heavy PDF documents contained large tables and screenshots that caused awkward page flow.	Rebuilt reports with more controlled sections, smaller tables, cover redesign, and fewer forced gaps.
Evidence consistency	Many screenshots were created across phases with varying naming.	Added evidence index, grouped phases, and evidence handling standards.
Supply-chain realism	Local lab provenance is useful but not equivalent to trusted CI-generated provenance.	Documented SLSA-style local provenance honestly and recommended trusted builder provenance as next step.
Intentionally vulnerable dependencies	The app intentionally used vulnerable packages to prove scanning.	Documented why vulnerable packages exist and separated lab risk from production readiness.

5. Timeline-Style Narrative

Phase	Activities	Outcome
Foundation	VM resource review, Docker setup, repository structure.	Stable private lab foundation.
Application security	Created vulnerable Flask routes; tested SQL injection, command injection, unsafe rendering.	Documented AppSec findings and remediation examples.
CI/CD	Added GitHub Actions, Bandit, pip-audit, Trivy, security gates, Scorecard.	Automated security validation story.
Kubernetes	Created hardened manifests, RBAC, NetworkPolicy, Pod Security, Trivy config scan.	Defensible workload deployment.
GitOps	Installed ArgoCD, created application, demonstrated drift self-healing.	Change integrity and self-heal proof.
Supply chain	Signed image, generated SBOM, attached attestations, documented provenance and exceptions.	Enterprise-grade supply-chain evidence.
Runtime detection	Installed Falco/Falcosidekick, simulated privileged pod and RBAC abuse, built dashboards.	Detection and response evidence.
Reporting	Generated professional PDFs, compliance mapping, evidence index, post-mortem.	Recruiter-ready project package.

6. Root Cause Analysis of Key Risks

Unsafe application code

The vulnerable Flask routes used unsafe patterns such as string-built SQL queries, shell=True command execution, and unsafe template rendering. These were intentionally introduced for lab validation, but in a production pipeline they would represent critical/high application security risks.

Overprivileged Kubernetes permissions

The RBAC attack simulation showed that a ClusterRoleBinding to cluster-admin gives excessive permissions. The corrected design used a namespace-scoped RoleBinding and validated secret access denial.

Privileged pod container escape pattern

The privileged hostPath pod demonstrated how workload isolation can fail. The preventive control was Pod Security restricted enforcement plus hardened workload manifests.

Supply-chain trust gaps

Before Cosign/Syft/provenance, image origin and contents were not strongly documented. Signing, SBOM generation, and attestations created a stronger trust story, while remaining honest about local-lab limitations.

7. Residual Risk and Accepted Limitations

Residual Risk	Why It Remains	Accepted / Next Action
Local lab is not production	Controls were implemented in Kind and private VM, not a managed production cluster.	Accepted for portfolio; next step is managed Kubernetes validation.
SLSA-style provenance is local	Local provenance lacks trusted builder guarantees.	Next step: GitHub Actions build provenance and immutable release tags.
Intentionally vulnerable app remains in repo	Vulnerabilities are required for demonstration.	Keep clear disclaimer and separate vulnerable branch from hardened branch.
Document review is manual	PDF quality depends on iterative visual review.	Continue using render-and-verify workflow before final delivery.
Compliance is alignment only	No formal audit scope, no regulated data, no third-party assessor.	Clearly mark as alignment-only and not certification.

8. Action Plan and Future Improvements

Priority	Improvement	Expected Benefit
High	Create hardened branch where SQL injection, command injection, and unsafe rendering are fixed.	Shows remediation beyond finding documentation.
High	Add branch protection requiring security-gates.yml and Scorecard workflows.	Strengthens CI/CD governance story.
High	Use immutable image tags and digest pinning in Kubernetes manifests.	Improves supply-chain integrity.
Medium	Add Kyverno or Sigstore Policy Controller for image signature verification at admission.	Turns signing evidence into deploy-time enforcement.
Medium	Add Loki or SIEM-style log storage for Falco events.	Improves incident investigation and retention.
Medium	Add a managed Kubernetes version of the lab.	Improves cloud-native realism.
Low	Create a short demo video walking through exploit, detection, remediation, and dashboard evidence.	Improves recruiter presentation.

9. Interview Narrative

This project can be explained as an end-to-end secure software delivery lab. The strongest narrative is: I built a deliberately vulnerable app, proved the flaws, built security gates, hardened the container and Kubernetes deployment, used GitOps for drift control, added supply-chain integrity with Cosign and

SBOMs, simulated Kubernetes attacks, detected them with Falco, monitored them with Prometheus/Grafana, and documented the work in audit-style reports.

10. Final Readiness Checklist

Readiness Area	Status / Action
GitHub README	Ensure the README includes architecture, tooling, screenshots, and disclaimer.
Evidence index	Keep the evidence index current with screenshot names and report artifacts.
Security findings	Link SQL injection, command injection, template injection, dependency, Kubernetes, and supply-chain reports.
Demo flow	Prepare a 5-7 minute walkthrough: vulnerable app, scan, hardening, GitOps, supply chain, runtime detection, dashboard.
Interview explanation	Be ready to explain why each tool was chosen and what control it validates.

11. Final Conclusion

The Enterprise DevSecOps Security Lab achieved its goal of demonstrating practical security engineering across the development, build, deployment, runtime, observability, and documentation lifecycle. The final project package is suitable for portfolio presentation, provided it is described honestly as a private lab and not as a formal compliance certification or production deployment.